

Initiation à Python

pour la data science

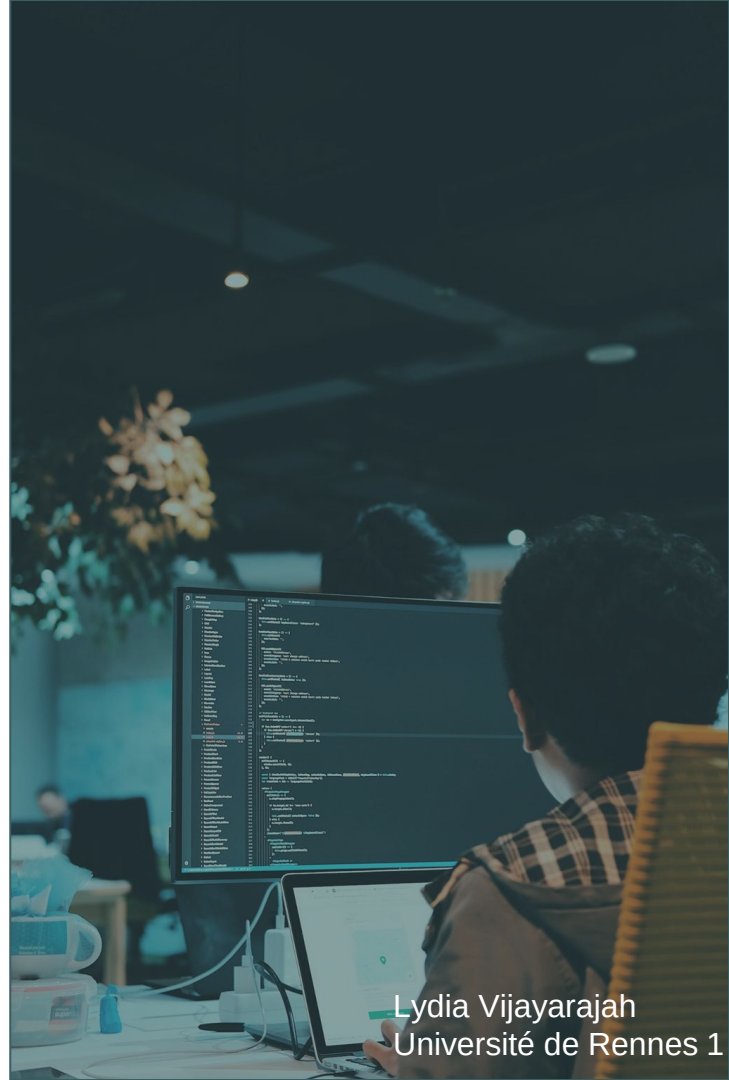
10 et 11 Septembre 2026

Lydia Vijayarajah
Université de Rennes 1

Initiation à Python

pour la data science

1. **Introduction** à Python
2. **Configuration** et **environnement** de Python
3. Les **jeux de données**
4. **Bonnes pratiques**
5. Principes de **programmation**
 - Les bases
 - Les structures de controle
 - Les classes



1. Introduction à Python

À propos de Python



- **langage informatique** de plus de **30 ans** dont le nom fait référence aux **Monty Python**
- 2 versions : **Python 2** et **Python 3 (prochaine version 3.15)**, Python 2 n'est plus maintenu
- **Communautés diverses** : statisticiens, développeurs, gestionnaires d'applications ou d'infrastructures, lycéens ou chercheurs dans des champs à la fois théoriques et appliqués.
- **Python** est utilisé dans **toutes les phases du traitement de la donnée**, de sa récupération et structuration à sa valorisation

Pourquoi utiliser Python pour la Data science ?

- fourni très tôt des **outils utiles** à l'entraînement d'algo **de machine learning** (Scikit Learn, Tensorflow, Pytorch)
- permet aussi la **récupération des données** via des APIs ou le web scrapping
- analyse de **données tabulaires**, publication de **contenu web** ou production de **graphique** de plus en plus poussé

2. Configuration et environnement de Python

Interpréteur Python

- **Installation de l'interpréteur** (le langage de programmation lui-même) Python :
 - a. Installation de Python en local pour exécuter votre code Python
 - b. Choix de la version : prendre la **version la plus récente** si possible
 - c. Ajouter Python au **PATH**
 - d. Installer **pip**
- A ce stade, ce n'est que le langage de base et un outil en ligne de commande

Environnement Python 1/2

- Un **socle de base** et des **packages** supplémentaires qui forment l'écosystème de Python
- Les **packages** communautaires sont **centralisés** sur des plateformes comme **PyPI**
- **Environnement** : ensemble des packages disponibles à Python pour effectuer des traitements spécifiques, à vous d'enrichir votre environnement en installant des packages (avec pip install)
- Utilisation de **venv** avec **poetry** ou uv

Environnement Python 2/2

- Il n'est pas pratique d'écrire directement son code dans la ligne de commande, il existe des **éditeurs** (appelés IDE)
- Les éditeurs sont des logiciels qui offrent une **interface pratique** pour écrire et exécuter
- **simplifier la lecture et l'écriture** de code: coloration syntaxique, autocomplétion, débuggage, diagnostics de qualité formelle du code, etc

```
Command Prompt
Microsoft Windows [Version 10.0.18362.113]
(c) 2019 Microsoft Corporation. All rights reserved.

C:\Windows>python3.7
Python 3.7.3 (tags/v3.7.3:ef4ec6d12, Mar 25 2019, 22:05:12) [MSC v.1916 64 bit (AMD64)]
Type "help", "copyright", "credits" or "license()" for more information.
>>> exit()

C:\Windows>pip3 list
Package Version
-----
certifi 2019.3.9
chardet 3.0.4
idna 2.8
requests 2.21.0
urllib3 1.24.2

C:\Windows>
```

```
import sys

# Simulate command-line arguments
sys.argv = ['program.py', '23', '10']

# Retrieve command-line arguments
arguments = sys.argv

# Check the number of arguments
if len(arguments) < 3:
    print("Insufficient arguments. Please provide at least two values.")
    sys.exit(1)

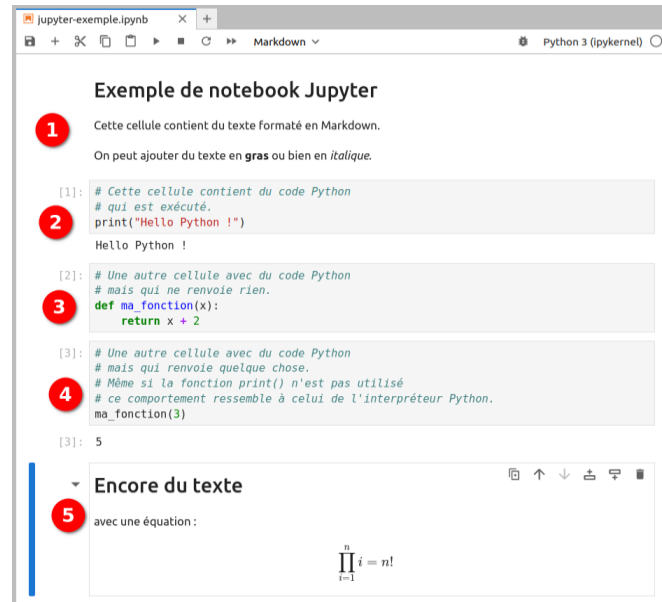
# Extract arguments
arg1 = arguments[1]
arg2 = arguments[2]

# Perform some operations based on the arguments
result = int(arg1) + int(arg2)
print("Result:", result)
```

Result: 33

Les notebooks Jupyter

- Offrent une **interface interactive** qui permet d'écrire du code Python, de le tester et d'avoir le résultat en dessous
- Simplifient grandement **l'exploration** et le **tâtonnement**
- Permettent de combiner du Markdown, du Python et du HTML
- VSCode sera à privilégier pour l'utilisation des notebooks



3. Les jeux de données

Récupération des données

- De quelles données ai-je **besoin** pour répondre à ma problématique ? La problématique amène à la réflexion sur les données disponibles plutôt que l'inverse
- Qui **produit** et **met à disposition** ces données ? Les sources disponibles sur sont-elles fiables ?
- Est-ce que je peux les **compléter** avec d'autres données ? Faire attention à avoir des niveaux de granularité adéquats

Structuration des données

- **Mise en forme** et **nettoyage** des jeux de données récupérés :
 - les informations **manquantes** sont bien comprises et traitées
 - les variables servant d'**identifiants** sont bien les mêmes d'une table à l'autre : même format, même modalités...
 - pour des variables textuelles, corriger les éventuelles fautes
 - créer des variables qui **synthétisent** l'information
 - **supprimer** les éléments inutiles (colonne ou ligne vide)
 - **renommer** les colonnes avec des noms compréhensibles

Analyse descriptive

Les questions à se poser pour “challenger” le jeu de données :

- Est-ce que mon échantillon est bien **représentatif** de ce qui m'intéresse ?
- Est-ce que les **ordres de grandeur** sont bons ?
- Est-ce que je comprends toutes les variables de mon jeu de données ? Est-ce qu'elles se “**comportent**” de la bonne façon ?
- Est-ce que j'ai des **outliers**, i.e. des valeurs aberrantes pour certains individus ?
- Est-ce que j'ai des premiers **grands messages** sortis de mon jeu de données ? Est-ce que j'ai des **résultats surprenants** ?

Selon les résultats, il est possible de devoir repasser sur le nettoyage des données

Modélisation

À cette étape, l'analyse descriptive doit avoir donné quelques **premières pistes** pour savoir dans quelle direction vous voulez mener votre modèle.

La méthode sera guidée par l'objectif et non l'inverse. Parmi les questions à considérer :

- Est-ce que vous voulez **expliquer ou prédire** ? Selon votre réponse à cette question, vous n'allez pas adopter la même approche scientifique ni les mêmes algorithmes.
- Est-ce que vous voulez **classer un élément dans une catégorie** (de manière supervisée via de la classification ou non supervisée avec du clustering) ou **prédire une valeur numérique** (régression) ?

Ethique et responsabilité

- La **reproductibilité** est importante
- Faire attention aux **biais cognitifs** : ne pas retenir que l'information qui correspond à nos conceptions a priori ou des représentations qui amènent à des interprétations erronées
- **Réglementation** des données : toujours faire attention au RGPD
- Partager les moyens de reproduire une analyse : **data-sharing**
- Adopter une approche **écologique** : les data scientists doivent être conscients des implications de leur usage intensif de ressources et essayer de minimiser leur impact
- Adopter une approche **inclusive**

4. Les bonnes pratiques de programmation

Résoudre les erreurs

- Bien lire les logs, i.e. les sorties renvoyées par Python en cas d'erreur. Souvent, elles sont informatives et peuvent contenir directement la réponse.
- Chercher sur internet (de préférence en Anglais et sur Google). Par exemple, donner le nom de l'erreur et une partie informative du message d'erreur renvoyé par Python
- Souvent, la recherche amènera vers le forum Stackoverflow, destiné à cet usage.
- Les documentations officielles (de Python et des différents packages) sont souvent un peu dures, mais généralement exhaustives.

Formatage du code

Utiliser un **linter** qui vise à améliorer la qualité du code. Le linter vérifie que la norme de nomenclature utilisée reste la même dans l'intégralité du package. Le package **pylint** permet de vérifier :

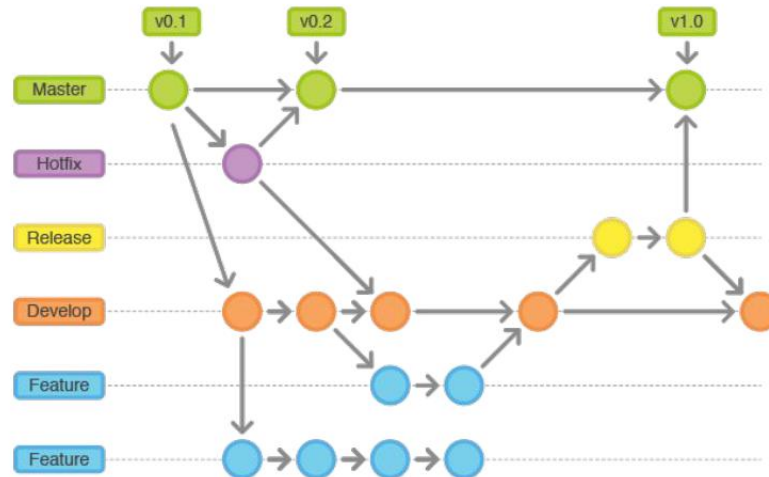
- Snake_case VS CamelCase
- L'indentation
- Les packages non utilisés
- L'appel à des variables non définies
- Les espaces autour des opérateurs
- La longueur des lignes

Black et Ruff permettent un formatage automatique avec VSCode

Utilisation de pydantic pour éviter les erreurs

Versionner son code avec GIT

- Afin d'éviter la perte de code, il est important de mettre son code sur GIT (<https://learngitbranching.js.org/>)
- Créer des branches spécifiques au versionnage de son code ou package



Tester son code

- Les tests unitaires sont essentiels à l'implémentation d'un package Python pérenne
- Utiliser le package test pour faciliter la création des fichiers de tests du package
- Une fois les tests effectués, s'assurer de la couverture des tests avec le package coverage. Celui-ci vérifie le pourcentage de lignes de code exécuté lors du lancement des tests du package.



Des questions ?

lydia.vijayarajah@gmail.com